

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра математического обеспечения и применения ЭВМ

ОТЧЕТ
по практической работе №4
по дисциплине «Машинное обучение»
Тема: Автоматизированный выбор и настройка моделей машинного
обучения с использованием Hyperopt-sklearn.

Студент гр. 1306

Шаврин А.П.

Преподаватель

Куликов И.А.

Санкт-Петербург

2025

Цель работы.

Изучение принципов автоматического выбора алгоритмов машинного обучения и настройки их гиперпараметров с использованием библиотеки Hyperopt-sklearn, а также приобретение практических навыков применения различных алгоритмов оптимизации для решения задач классификации и анализа качества полученных моделей.

Основные теоретические положения.

Hyperopt-sklearn - это программная библиотека на языке Python, предназначенная для автоматического выбора алгоритмов машинного обучения и настройки их гиперпараметров. Данная библиотека реализует подход AutoML и является связующим звеном между библиотекой scikit-learn, предоставляющей широкий набор алгоритмов машинного обучения, и библиотекой Hyperopt, отвечающей за оптимизацию гиперпараметров.

Оптимизация гиперпараметров в Hyperopt-sklearn осуществляется путём поиска по заданному пространству гиперпараметров. Для поиска могут использоваться различные алгоритмы оптимизации, включая случайный поиск, метод имитации отжига, дерево оценок Парзена (TPE) и его адаптивную модификацию (ATPE). Выбор конкретного алгоритма поиска задаётся пользователем при инициализации процесса оптимизации.

Для выполнения экспериментов данные могут быть загружены как из стандартных наборов данных библиотеки scikit-learn, так и из открытого репозитория OpenML, а также из внешних файлов. После загрузки данные представляются в виде структур, содержащих описание набора данных, матрицу признаков, целевую переменную и вспомогательную информацию. В случае использования OpenML данные, как правило, загружаются в формате DataFrame библиотеки pandas.

Основным инструментом работы с Hyperopt-sklearn является класс HyperoptEstimator. Он позволяет автоматически подбирать классификатор, методы предварительной обработки данных и значения гиперпараметров. Пользователь может использовать параметры по умолчанию либо явно задать пространство поиска классификаторов, методов предобработки и алгоритм оптимизации. Дополнительно можно ограничить количество итераций поиска или время выполнения каждой итерации.

После завершения процесса оптимизации становится доступна информация о найденной лучшей модели, включая тип классификатора и оптимальные значения его гиперпараметров. Полученная модель может быть использована для предсказания на новых данных и оценки качества классификации. Для анализа результатов работы классификатора применяется показатель точности, а также матрица ошибок, позволяющая наглядно оценить качество классификации для каждого класса.

Таким образом, библиотека Hyperopt-sklearn предоставляет удобный инструмент для автоматизированного подбора моделей машинного обучения и позволяет упростить процесс экспериментального сравнения различных алгоритмов и их параметров при решении задач классификации и регрессии.

Постановка задачи.

В рамках лабораторной работы требуется исследовать возможности автоматизированного машинного обучения с использованием Hyperopt-sklearn при решении трех практических задач.

Задача 1.

Исследовать стандартный набор данных `digit` содержащий информацию о числах. Данный набор загружается с помощью метода `load_digits` библиотеки `sklearn`.

1. Загрузить набор данных

2. Разбить на обучающую и тестовую часть.
3. С помощью Hyperopt-sklearn выбрать подходящий классификатор. Проверять все классификаторы, использовать для поиска дерево оценок Парзена.
4. Используя тестовые данные вывести оценку точности и матрицу ошибок.
5. Использовать для выбора классификатора объект класса HyperoptEstimator без параметров (с параметрами по умолчанию). Сравнить результаты.

Задача 2.

Исследовать набор данных №151 из репозитория OpenML, который содержит информацию об энергопотреблении и о колебаниях цен на электричество.

1. Загрузить набор данных из репозитория.
2. Вывести на экран описание этого набора данных.
3. Разбить на обучающую и тестовую части.
4. С помощью Hyperopt-sklearn выбрать подходящий классификатор, применяя для поиска такие же параметры, как в предыдущем задании.
5. Используя тестовые данные вывести оценку точности и матрицу ошибок.
6. С помощью Hyperopt-sklearn выбрать подходящий классификатор. Проверять все классификаторы, использовать смешанный алгоритм поиска: 1/4 случайный поиск, 1/4 – отжиг, 1/2 – дерево оценок Парзена. Сравнить результаты с предыдущим пунктом.
7. Использовать на тестовых данных предложенный классификатор, но без параметров (с параметрами по умолчанию). Сравнить результаты.

Задание 3.

Исследовать набор данных из файла creditcard.arff, содержащий информацию о транзакциях по кредитным картам.

1. Загрузить набор данных из файла.
2. Преобразовать загруженные данные к необходимому виду.
3. Разбить на обучающую и тестовую части.

4. С помощью Hyperopt-sklearn выбрать подходящий классификатор. Проверить все классификаторы, для поиска использовать адаптивное дерево оценок Парзена.
5. Используя тестовые данные вывести оценку точности и матрицу ошибок.

Выполнение работы.

1. Задание 1.

Набор данных digits загружается функцией `load_digits()` из `sklearn.datasets`. Данные представлены матрицей признаков X размерности (1797, 64), где каждая строка соответствует изображению 8×8 , развернутому в вектор длины 64, и вектором целевых меток y (0–9). Информация о размерах датасета (рис 1)

```
=== DIGITS DATASET ===  
X shape: (1797, 64)  
y shape: (1797,)
```

Рисунок 1 - Информации о наборе данных digits (размерности X и y).

Разбиение данных выполняется с помощью функции `train_test_split` из `sklearn.model_selection`. В работе используется соотношение 80/20 (`test_size=0.2`) и фиксированный `seed` для воспроизводимости эксперимента.

Для выполнения требований задания пространство классификаторов задается через `any_classifier("clf")`, что позволяет Hyperopt-sklearn рассмотреть все доступные классификаторы. Пространство предобработок задается `any_preprocessing("prep")`. В качестве алгоритма оптимизации используется `tpe.suggest`. После обучения (`estim.fit`) выполняется предсказание на тестовых данных (`estim.predict`) и вычисляется точность (`accuracy_score`). Для визуального анализа ошибок строится матрица ошибок Confusion Matrix (`ConfusionMatrixDisplay.from_predictions`). В рамках работы матрица сохраняется в файл `cm_digits_tpe.png`. Результат обучения представлен на рисунке 2, а матрица ошибок на рисунке 3.

```

=== DIGITS DATASET ===
x shape: (1797, 64)
y shape: (1797,)
100% | 1/1 [00:00:00:00, 3.39trial/s, best loss: 0.08333333333333337]
100% | 2/2 [00:00:00:00, 8.58trial/s, best loss: 0.03819444444444442]
100% | 3/3 [00:00:00:00, 1.22trial/s, best loss: 0.03819444444444442]
100% | 4/4 [00:00:00:00, 10.44trial/s, best loss: 0.03819444444444442]
100% | 5/5 [00:00:00:00, 3.57trial/s, best loss: 0.03819444444444442]
100% | 6/6 [00:00:00:00, 2.35trial/s, best loss: 0.03819444444444442]
100% | 7/7 [00:00:00:00, 1.22trial/s, best loss: 0.03819444444444442]
100% | 8/8 [00:00:00:00, 3.77trial/s, best loss: 0.03819444444444442]
100% | 9/9 [00:00:00:00, 10.05trial/s, best loss: 0.03819444444444442]
100% | 10/10 [00:00:00:00, 5.70trial/s, best loss: 0.03819444444444442]

=== METRICS ===
Accuracy: 0.9861111111111112

=== BEST MODEL ===
{'learner': SVC(C=1.247410264410966, coef0=0.87441768104555, decision_function_shape='ovr',
degree=5, gamma='auto', kernel='linear', random_state=np.int64(3),
tol=0.0013056724208330755), 'preprocs': (MinMaxScaler(feature_range=(0.0, 1.0)),), 'ex_preprocs': ()}

```

Рисунок 2 - Точность классификации (Accuracy) для конфигурации Hyperopt-sklearn (TPE)

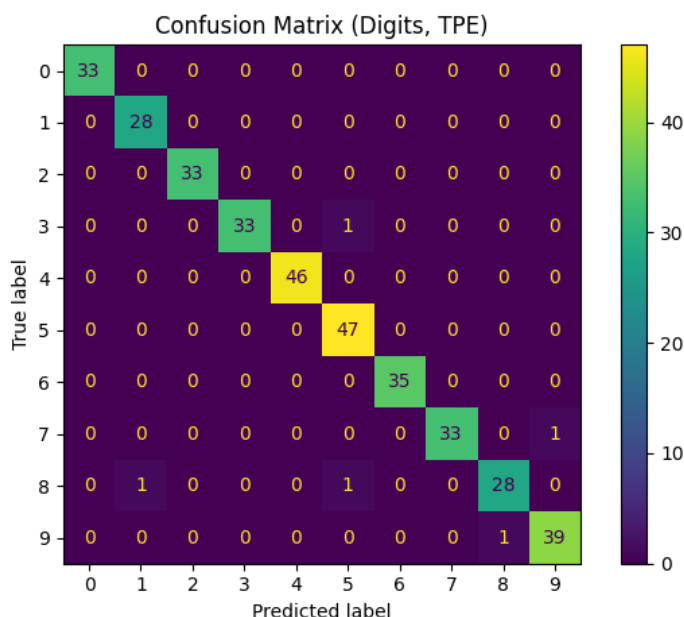


Рисунок 3 - Матрица ошибок для лучшей конфигурации Hyperopt-sklearn (TPE)

Для сравнения выполняется запуск HyperoptEstimator() без явного задания classifier/preprocessing/algo. В этом режиме используются параметры по умолчанию библиотеки. Далее повторяются этапы предсказания и оценки на тестовой выборке, а матрица ошибок сохраняется в файл cm_digits_default.png. Результат обучения представлен на рисунке 4, а матрица ошибок на рисунке 5.

```

100% | 1/1 [00:01:00:00, 1.48s/trial, best loss: 0.8333333333333334]
100% | 2/2 [00:00:00:00, 4.60trial/s, best loss: 0.71875]
100% | 3/3 [00:00:00:00, 11.68trial/s, best loss: 0.71875]
100% | 4/4 [00:00:00:00, 1.27trial/s, best loss: 0.04861111111111116]
100% | 5/5 [00:00:00:00, 3.26trial/s, best loss: 0.04861111111111116]
100% | 6/6 [00:00:00:00, 3.49trial/s, best loss: 0.02777777777777777]
100% | 7/7 [00:00:00:00, 3.45trial/s, best loss: 0.02777777777777777]
100% | 8/8 [00:00:00:00, 2.22trial/s, best loss: 0.02777777777777777]
100% | 9/9 [00:00:00:00, 11.08trial/s, best loss: 0.02777777777777777]
100% | 10/10 [00:00:00:00, 5.12trial/s, best loss: 0.02777777777777777]

=== DEFAULT METRICS ===
Accuracy: 0.9888888888888889

=== BEST DEFAULT MODEL ===
{'learner': KNeighborsClassifier(algorithm='brute', leaf_size=25, metric='euclidean',
n_jobs=1, n_neighbors=7, p=3.055724972534203), 'preprocs': (Normalizer(norm='max'),), 'ex_preprocs': ()}

```

Рисунок 4 - Точность классификации (Accuracy) для конфигурации Hyperopt-sklearn по умолчанию

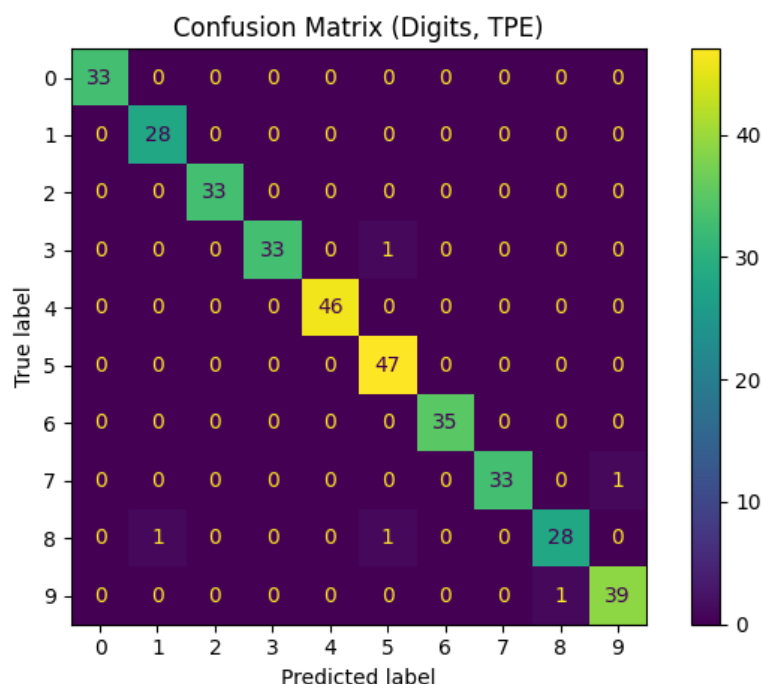


Рисунок 5 - Матрица ошибок для конфигурации HyperoptEstimator по умолчанию.

В результате проведенного эксперимента с набором данных Digits был выбран оптимальный классификатор с помощью Hyperopt-sklearn как с использованием кастомного `any_classifier`, так и с использованием объекта `HyperoptEstimator` без параметров по умолчанию. В первом случае точность составила 0.986, во втором - 0.989. Разница в результатах невелика и может быть обусловлена случайностью процесса оптимизации гиперпараметров, особенностями алгоритма TPE и различиями в комбинации классификатора и методов препроцессинга, которые были исследованы в каждом запуске.

2. Задание 2.

Набор данных загружается с использованием функции `fetch_openml()` из `sklearn.datasets`. По умолчанию данные возвращаются в виде `pandas.DataFrame` (параметр `as_frame=True`). Целевая переменная `dataset.target` кодируется в целочисленный вид с помощью `LabelEncoder`, что требуется большинству классификаторов `scikit-learn`. Описание набора данных выводится через поле `dataset.DESCR` (рисунок 6). Оно содержит сведения о происхождении данных, составе признаков, описании классов и рекомендуемом способе использования.

```

=== DESCRIPTION ===
**Author**: M. Harries, J. Gama, A. Bifet
**Source**: [Joao Gama](http://www.inescporto.pt/~jgama/ales/ales_5.html) - 2009
**Please cite**: None

**Electricity** is a widely used dataset described by M. Harries and analyzed by J. Gama (see papers below). This data was collected from the Australian New South Wales Electricity Market. In this market, prices are not fixed and are affected by demand and supply of the market. They are set every five minutes. Electricity transfers to/from the neighboring state of Victoria were done to alleviate fluctuations.

The dataset (originally named ELEC2) contains 45,312 instances dated from 7 May 1996 to 5 December 1998. Each example of the dataset refers to a period of 30 minutes, i.e. there are 48 instances for each time period of one day. Each example on the dataset has 5 fields, the day of week, the time stamp, the New South Wales electricity demand, the Victoria electricity demand, the scheduled electricity transfer between states and the class label. The class label identifies the change of the price (UP or DOWN) in New South Wales relative to a moving average of the last 24 hours (and removes the impact of longer term price trends).

The data was normalized by A. Bifet.

=== Attribute information
* Date: date between 7 May 1996 to 5 December 1998. Here normalized between 0 and 1
* Day: day of the week (1-7)
* Period: time of the measurement (1-48) in half hour intervals over 24 hours. Here normalized between 0 and 1
* NSWprice: New South Wales electricity price, normalized between 0 and 1
* NSWdemand: New South Wales electricity demand, normalized between 0 and 1
* Vicprice: Victoria electricity price, normalized between 0 and 1
* Viddemand: Victoria electricity demand, normalized between 0 and 1
* transfer: scheduled electricity transfer between both states, normalized between 0 and 1

=== Relevant papers
M. Harries. Splice-2 comparative evaluation: Electricity pricing. Technical report, The University of South Wales, 1999.
J. Gama, P. Medas, G. Castillo, and P. Rodrigues. Learning with drift detection. In SBIA Brazilian Symposium on Artificial Intelligence, pages 286-295, 2004.

```

Рисунок 6 – Описание набора данных.

Разбиение выполняется функцией `train_test_split` из `sklearn.model_selection`. Используется соотношение 80/20 (`test_size=0.2`) и фиксированный `seed` (`random_state=42`) для воспроизводимости.

Для выполнения требований задания пространство классификаторов задается через `any_classifier('clf')`, что позволяет Hyperopt-sklearn рассмотреть все доступные классификаторы. Пространство предобработок задается `any_preprocessing('prer')`. В качестве алгоритма оптимизации используется `tre.suggest`. После обучения (`estim.fit`) выполняется предсказание на тестовых данных (`estim.predict`) и вычисляется точность (`accuracy_score`). Для визуального анализа ошибок строится матрица ошибок Confusion Matrix (`ConfusionMatrixDisplay.from_predictions`). Матрица ошибок сохраняется в файл `cm_electricity_tpe_2.png`. Результат обучения представлен на рисунке 7, а матрица ошибок на рисунке 8.

```

100%|#####| 1/1 [00:00<00:00, 2.11trial/s, best loss: 0.22482758620689658]
50%|#####| 1/2 [00:00<?, ?trial/s, best loss=?]
/home/aleksei/Development/LetiMaster/1_semester/AutoMachineLearning/lab4/.venv/lib/python3.11/site-packages/sklearn/linear_model/_stochastic_gradient.py:726: ConvergenceWarning: Maximum number of iteration reached before convergence. Consider increasing max_iter to improve the fit.
  warnings.warn(

100%|#####| 2/2 [00:03<00:00, 3.62s/trial, best loss: 0.22482758620689658]
100%|#####| 3/3 [00:06<00:00, 5.62trial/s, best loss: 0.22482758620689658]
100%|#####| 4/4 [00:08<00:00, 2.78trial/s, best loss: 0.19793103448275862]
100%|#####| 5/5 [00:11<00:00, 1.61s/trial, best loss: 0.19793103448275862]
100%|#####| 6/6 [00:14<00:00, 1.02s/trial, best loss: 0.19793103448275862]
100%|#####| 7/7 [00:17<00:00, 5.58trial/s, best loss: 0.19793103448275862]
100%|#####| 8/8 [00:20<00:00, 2.81trial/s, best loss: 0.19793103448275862]
100%|#####| 9/9 [00:23<00:00, 1.38s/trial, best loss: 0.19635172413793103]
100%|#####| 10/10 [01:24<00:00, 89.54s/trial, best loss: 0.19635172413793103]
/home/aleksei/Development/LetiMaster/1_semester/AutoMachineLearning/lab4/.venv/lib/python3.11/site-packages/sklearn/utils/validation.py:2749: UserWarning: X does not have valid feature names, but MinMaxScaler was fitted with feature names
  warnings.warn(

=== TPE RESULTS ===
Accuracy: 0.809996689837802

=== BEST MODEL ===
{'learner': 'KNeighborsClassifier(algorithm='brute', leaf_size=35, metric='manhattan',
n_jobs=1, n_neighbors=4, p=3.7683377637996274), 'preprocs': (MinMaxScaler(clone=True, feature_range=(-1.0, 1.0)),), 'ex_preprocs': ()}

```

Рисунок 7 - Точность классификации (Accuracy) для конфигурации Hyperopt-sklearn (TPE)

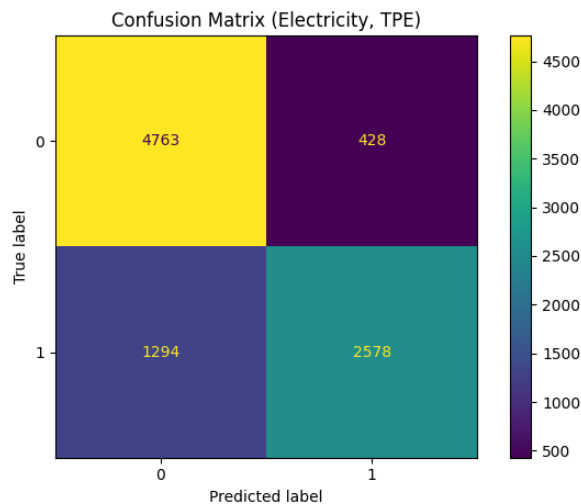


Рисунок 8 - Матрица ошибок для конфигурации HyperoptEstimator (TPE).

Для сравнения выполняется запуск Hyperopt-sklearn со смешанной стратегией поиска. Используется комбинация: 1/4 случайный поиск (rand), 1/4 имитация отжига (anneal), 1/2 - TPE. Смешивание реализуется функцией `mix.suggest` с указанием весов `p_suggest`. Остальные параметры (пространства classifier/preprocessing) задаются аналогично предыдущему пункту. Результат обучения представлен на рисунке 9, а матрица ошибок на рисунке 10.

```
100% | 1/1 [01:01:00:00, 61.91s/trial, best loss: 0.17096551724137932]
100% | 2/2 [00:01:00:00, 1.50s/trial, best loss: 0.17020689655172416]
100% | 3/3 [00:00:00:00, 3.60s/trial, best loss: 0.1424827586206897]
100% | 4/4 [00:38:00:00, 38.10s/trial, best loss: 0.1424827586206897]
100% | 5/5 [00:00:00:00, 1.25s/trial, best loss: 0.1424827586206897]
100% | 6/6 [00:01:00:00, 1.20s/trial, best loss: 0.1424827586206897]
100% | 7/7 [00:00:00:00, 1.33s/trial, best loss: 0.1424827586206897]
100% | 8/8 [00:02:00:00, 2.55s/trial, best loss: 0.1424827586206897]
100% | 9/9 [00:02:00:00, 2.56s/trial, best loss: 0.1424827586206897]
100% | 10/10 [00:04:00:00, 4.65s/trial, best loss: 0.09696551724137936]

/home/alexsi/development/latiMaster/1_semester/AutoMachineLearning/lab4/venv/lib/python3.11/site-packages/sklearn/utils/validation.py:2769: UserWarning: X does not have valid feature names, but MinMaxScaler was fitted
  warnings.warn(

=== MIXED SEARCH RESULTS ===
Accuracy: 0.9142667990731546

=== BEST MODEL ===
{'learner': RandomForestClassifier(bootstrap=False, class_weight='balanced_subsample',
  criterion='entropy', n_estimators=45, n_jobs=1,
  random_state=np.int64(1), verbose=False), 'preprocs': (MinMaxScaler(feature_range=(-1.0, 1.0)),), 'ex_preprocs': ()}
```

Рисунок 9 - Точность классификации (Ассурасу) для конфигурации Hyperopt-sklearn (MIXED)

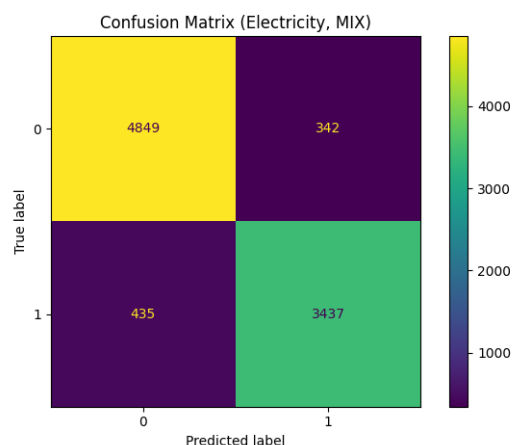


Рисунок 10 - Матрица ошибок для конфигурации HyperoptEstimator (MIXED).

Для выполнения пункта 7 задания используется алгоритм, предложенный Hyperopt-sklearn (по результатам смешанного поиска), но запускается с параметрами по умолчанию. Для этого берется тип лучшего классификатора (`learner.__class__`) и создается новый экземпляр без явной передачи гиперпараметров. Матрица ошибок для данного эксперимента сохраняется в файл `cm_electricity_mix_default.png`. Результат обучения представлен на рисунке 11, а матрица ошибок на рисунке 12.

```
=== ESTIMATOR ===  
Accuracy: 0.9048880061789695
```

Рисунок 11 - Точность классификации (Ассурасу) для классификатора предыдущего пункта с гиперпараметрами по умолчанию

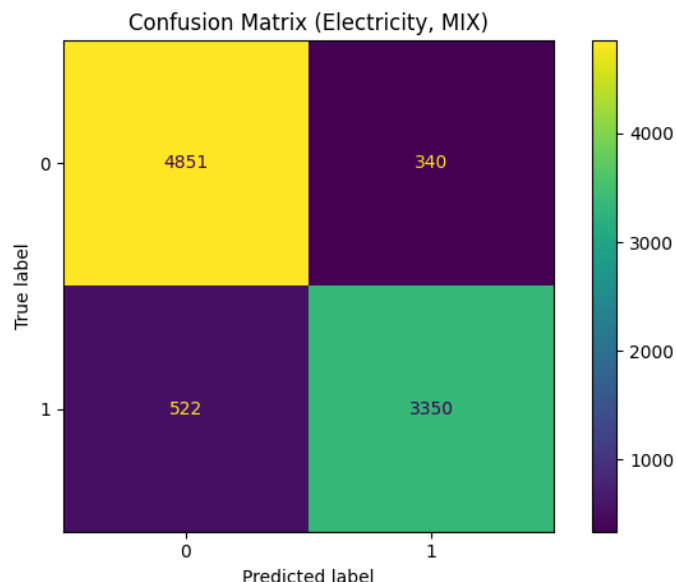


Рисунок 12 - Матрица ошибок для конфигурации HyperoptEstimator по умолчанию.

На основе приведенных результатов можно сделать следующие наблюдения и выводы:

Разные алгоритмы подбора гиперпараметров дают разную точность. При использовании ТРЕ точность составила ≈ 0.81 , тогда как при смешанном поиске (комбинация ТРЕ, отжиг и случайного поиска) удалось достичь ≈ 0.91 . Это демонстрирует, что комбинированные стратегии поиска позволяют более

эффективно исследовать пространство гиперпараметров и находить модели с лучшей обобщающей способностью.

Оптимизация гиперпараметров действительно улучшает результат. Классический RandomForestClassifier без настройки гиперпараметров показал точность ≈ 0.90 . После подбора гиперпараметров через смешанный поиск точность повысилась до ≈ 0.91 . Разница не огромная, но статистически значимая — настройка параметров (количество деревьев, критерий разбиения, балансировка классов) позволяет модели лучше адаптироваться к конкретным особенностям данных.

3. Задание 3

Для загрузки данных используется формат ARFF (Attribute-Relation File Format).

Файл читается функцией: `arff.loadarff(file_path)` — загружает массив записей и метаданные, `df = pd.DataFrame(data)` — преобразует данные в табличный вид `pandas.DataFrame`. После загрузки выводится структура данных (рисунок 13 и 14): `df.info()` — показывает число строк, типы столбцов, пропуски.

Затем данные делятся на:

`X = df.iloc[:, :-1]` — все признаки (все столбцы, кроме последнего),

`y = df.iloc[:, -1]` — целевая переменная (последний столбец).

Разбиение выполняется функцией `train_test_split` из `sklearn.model_selection`. Используется соотношение 80/20 (`test_size=0.2`) и фиксированный `seed` (`random_state=42`) для воспроизводимости.

Для выполнения требований задания пространство классификаторов задается через `any_classifier("clf")`, что позволяет Hyperopt-sklearn рассмотреть все доступные классификаторы. Пространство предобработок задается `any_preprocessing("prep")`. В качестве алгоритма оптимизации используется `tre.suggest`. После обучения (`estim.fit`) выполняется предсказание на тестовых данных (`estim.predict`) и вычисляется точность (`accuracy_score`). Для визуального анализа ошибок строится матрица ошибок Confusion Matrix (`ConfusionMatrixDisplay.from_predictions`). В рамках работы матрица

сохраняется в файл `cm_digits_tpe_3.png`. Результат обучения представлен на рисунке 15, а матрица ошибок на рисунке 16.

```
=== CREDIT CARD DATASET ===

=== INFO ===
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 284807 entries, 0 to 284806
Data columns (total 31 columns):
#   Column      Non-Null Count  Dtype
---  -
0    Time      284807 non-null  float64
1    V1         284807 non-null  float64
2    V2         284807 non-null  float64
3    V3         284807 non-null  float64
4    V4         284807 non-null  float64
5    V5         284807 non-null  float64
6    V6         284807 non-null  float64
7    V7         284807 non-null  float64
8    V8         284807 non-null  float64
9    V9         284807 non-null  float64
10   V10        284807 non-null  float64
11   V11        284807 non-null  float64
12   V12        284807 non-null  float64
13   V13        284807 non-null  float64
14   V14        284807 non-null  float64
15   V15        284807 non-null  float64
16   V16        284807 non-null  float64
17   V17        284807 non-null  float64
18   V18        284807 non-null  float64
19   V19        284807 non-null  float64
20   V20        284807 non-null  float64
21   V21        284807 non-null  float64
22   V22        284807 non-null  float64
23   V23        284807 non-null  float64
24   V24        284807 non-null  float64
25   V25        284807 non-null  float64
26   V26        284807 non-null  float64
27   V27        284807 non-null  float64
28   V28        284807 non-null  float64
29   Amount     284807 non-null  float64
30   Class      284807 non-null  float64
dtypes: float64(31)
```

Рисунок 13 – Информация о наборе данных.

	Time	V1	V2	V3	V4	V5	V6	...	V24	V25	V26	V27	V28	Amount	Class
count	284807.000000	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	...	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	2.848070e+05	284807.000000	284807.000000
mean	94813.859575	1.175161e-15	3.480941e-16	-1.379537e-15	2.894852e-15	1.021879e-15	1.494498e-15	...	4.473110e-15	5.109395e-16	1.686180e-15	-3.661401e-16	-1.217473e-16	88.349619	0.801727
std	47488.145955	1.986990e+00	1.651399e+00	1.516255e+00	1.415809e+00	1.380247e+00	1.332271e+00	...	6.856471e-01	5.212781e-01	4.822270e-01	4.836325e-01	3.308833e-01	250.120109	0.841527
min	0.000000	-5.440751e+01	-7.271573e+01	-4.812559e+01	-5.463171e+00	-1.137433e+02	-2.616851e+01	...	-2.836627e+00	-1.029540e+01	-2.404551e+00	-2.356560e+01	-1.543088e+01	0.000000	0.000000
25%	54281.500000	-9.203734e-01	-5.985499e-01	-8.903648e-01	-8.486401e-01	-6.915971e-01	-7.682955e-01	...	-3.545861e-01	-3.171451e-01	-3.269839e-01	-7.083953e-02	-5.295979e-02	5.400000	0.000000
50%	84692.000000	1.810880e-02	6.548556e-02	1.798463e-01	-1.984653e-02	-5.433583e-02	-2.741871e-01	...	4.897608e-02	1.659350e-02	-5.213911e-02	1.342146e-03	1.124383e-02	22.000000	0.000000
75%	139320.500000	1.315642e+00	8.037239e-01	1.027196e+00	7.433413e-01	6.119264e-01	3.985049e-01	...	4.395266e-01	3.507156e-01	2.409522e-02	9.104512e-02	7.827995e-02	77.165000	0.000000
max	172792.000000	2.454930e+00	2.205773e+01	9.382558e+00	1.687534e+01	3.480167e+01	7.330163e+01	...	4.584549e+00	7.519589e+00	3.517346e+00	3.161220e+01	3.384781e+01	25691.160000	1.000000

Рисунок 14 – Информация о наборе данных 2.

```
None
100% | 1/1 [00:02<00:00, 2.92s/trial, best loss: 0.000746121266505235]
100% | 2/2 [00:06<00:00, 6.19s/trial, best loss: 0.000746121266505235]
100% | 3/3 [01:40<00:00, 180.19s/trial, best loss: 0.000746121266505235]
100% | 4/4 [01:40<00:00, 180.17s/trial, best loss: 0.000746121266505235]
100% | 5/5 [00:26<00:00, 26.43s/trial, best loss: 0.000746121266505235]
100% | 6/6 [00:08<00:00, 8.04s/trial, best loss: 0.000746121266505235]
100% | 7/7 [01:40<00:00, 180.26s/trial, best loss: 0.000746121266505235]
100% | 8/8 [00:11<00:00, 11.82s/trial, best loss: 0.000746121266505235]
100% | 9/9 [00:35<00:00, 35.11s/trial, best loss: 0.000746121266505235]
100% | 10/10 [02:10<00:00, 10.17s/trial, best loss: 0.000746121266505235]
/home/aleksei/Development/LetiMaster/1_semester/AutoMachineLearning/lab4/.venv/lib/python3.11/site-packages/sklearn/utils/validation.py:2749: UserWarning: X does not have valid feature names, but MinMaxScaler was fitted
  warnings.warn(

with feature names
warnings.warn(

=== METRICS ===
Accuracy: 0.9991924440855307

=== BEST MODEL ===
{'learner': ExtraTreeClassifier(criterion='entropy', max_features=0.1002118306720676,
                               random_state=np.int64(4), splitter='best'), 'preprocs': (MinMaxScaler(feature_range=(-1.0, 1.0)),), 'ex_preprocs': ()}
```

Рисунок 15 - Точность классификации (Accuracy) для конфигурации Hyperopt-sklearn (TPE)

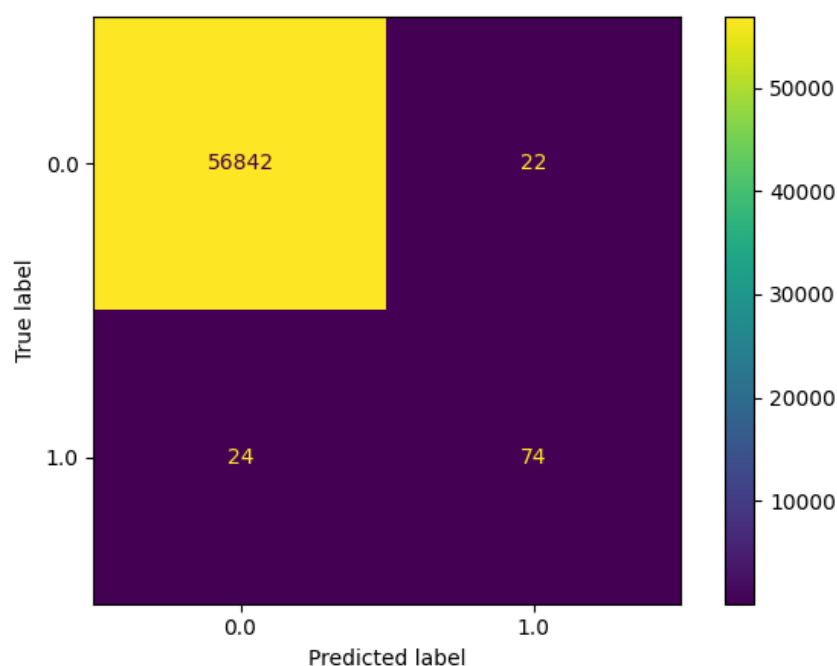


Рисунок 16 - Матрица ошибок для конфигурации HyperoptEstimator (TPE).

По результатам запуска Hyperopt-sklearn на наборе транзакций кредитных карт модель была подобрана автоматически и показала очень высокую точность на тестовой выборке (Accuracy = 0.99919). Лучшей конфигурацией оказался классификатор ExtraTreeClassifier с критерием разбиения entropy и ограничением числа учитываемых признаков при разбиении до примерно 10% ($\text{max_features} \approx 0.1002$), а также с предобработкой MinMaxScaler, приводящей признаки к диапазону $[-1; 1]$.

Всего наблюдений: 56 962, из них “положительных” (класс 1) всего 98 ($\approx 0.172\%$) — выборка сильно несбалансирована, поэтому высокая Accuracy сама по себе не является достаточной характеристикой.

Ключевые метрики для класса 1:

Accuracy = 0.99919 ($\approx 99.92\%$) — высокая, но главным образом за счёт доминирования класса 0.

Precision (класс 1) = $74 / (74 + 22) = 0.7708$ ($\approx 77.1\%$): из всех срабатываний около 23% — ложные.

$\text{Recall (класс 1)} = 74 / (74 + 24) = 0.7551 (\approx 75.5\%)$: модель находит примерно 3/4 мошеннических операций, но $\approx 24.5\%$ мошенничества пропускает (FN).

$\text{F1-score (класс 1)} \approx 0.7629$ — баланс точности и полноты.

$\text{Specificity (класс 0)} = 56\,842 / (56\,842 + 22) = 0.99961 (\approx 99.96\%)$: ложных тревог крайне мало.

$\text{Balanced accuracy} \approx 0.8774$ — более информативна при дисбалансе; показывает, что качество по редкому классу заметно ниже, чем по большинству.

Модель почти безошибочно распознаёт нормальные транзакции (очень высокая специфичность и крайне низкий FP), однако по мошенничеству качество умеренное: примерно четверть мошеннических операций остаётся невыявленной (FN=24). Если в задаче критичнее минимизировать пропуски мошенничества, имеет смысл смещать модель в сторону большего Recall (например, настройкой порога, cost-sensitive подходом, балансировкой классов), принимая возможный рост числа ложных срабатываний.

Выводы.

В ходе выполнения лабораторной работы были изучены и практически освоены принципы автоматизированного выбора алгоритмов машинного обучения и настройки их гиперпараметров с использованием библиотеки Hyperopt-sklearn. Реализован полный экспериментальный цикл для трех задач классификации: загрузка и первичный анализ данных, разбиение на обучающую и тестовую выборки, запуск автоматизированного подбора модели и гиперпараметров с заданными стратегиями оптимизации, а также оценка качества полученных решений по метрике точности и с использованием матриц ошибок.

На наборе данных Digits продемонстрирована работоспособность подхода AutoML при переборе различных моделей и методов предобработки с использованием TPE. Полученные значения точности при заданной конфигурации поиска и при использовании HyperoptEstimator с параметрами по

умолчанию оказались сопоставимыми (0.986 и 0.989), что свидетельствует о стабильности решения на относительно небольшом и хорошо подготовленном наборе данных, а также о том, что даже базовые настройки AutoML способны давать конкурентоспособные результаты. Небольшая разница между запусками объясняется стохастической природой оптимизации и различиями в исследованных комбинациях моделей и препроцессинга.

В задаче с набором OpenML №151 показано существенное влияние выбранной стратегии поиска по пространству гиперпараметров на итоговое качество. Использование TPE обеспечило точность порядка 0.81, тогда как применение смешанного алгоритма (rand/anneal/TPE) позволило повысить результат до ≈ 0.91 . Это подтверждает, что комбинирование различных методов оптимизации может улучшать исследование пространства решений и повышать качество итоговой модели. Дополнительно был выполнен контрольный эксперимент, в котором предложенный Hyperopt-sklearn алгоритм запускался с параметрами по умолчанию, что позволило отделить вклад выбора архитектуры модели от вклада тонкой настройки гиперпараметров и показать, что оптимизация параметров дает измеримое, хотя и умеренное улучшение.

В задаче анализа транзакций по кредитным картам (creditcard.arff) автоматизированный подбор привел к выбору модели ExtraTreeClassifier с предобработкой MinMaxScaler и обеспечил очень высокую точность на тестовой выборке (Accuracy = 0.99919). При этом результат иллюстрирует типичную особенность задач с сильной несбалансированностью классов. Модель почти безошибочно распознаёт нормальные транзакции, однако по мошенничеству качество умеренное: примерно четверть мошеннических операций остаётся невыявленной.

Таким образом, лабораторная работа показала, что Hyperopt-sklearn является удобным инструментом для автоматизированного подбора моделей и их параметров, позволяет быстро сравнивать различные алгоритмы и стратегии оптимизации и получать конкурентоспособные решения без ручного перебора. Эксперименты подтвердили, что качество итоговой модели зависит как от

выбора алгоритма, так и от настройки гиперпараметров, а выбор стратегии поиска может существенно влиять на результат и вычислительные затраты.

ПРИЛОЖЕНИЕ А

ИСХОДНЫЙ КОД

main.py

```
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.datasets import load_digits, fetch_openml
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, ConfusionMatrixDisplay

from hyperopt import tpe, rand, anneal, partial, mix, hp
from hpsklearn import (HyperoptEstimator, any_preprocessing,
any_classifier)
from hpsklearn import (
    gradient_boosting_classifier, svc, random_forest_classifier,
k_neighbors_classifier, extra_tree_classifier, ada_boost_classifier,
sgd_classifier,
)
from sklearn.preprocessing import LabelEncoder
from scipy.io import arff

def custom_any_classifier(name):
    classifiers = [
        svc(f"{name}.svc"),
        k_neighbors_classifier(f"{name}.knn"),
        random_forest_classifier(f"{name}.random_forest"),
        extra_tree_classifier(f"{name}.extra_trees"),
        ada_boost_classifier(f"{name}.ada_boost"),
        gradient_boosting_classifier(f"{name}.grad_boosting"),
loss="log_loss"),
        sgd_classifier(f"{name}.sgd"),
    ]

    return hp.choice(name, classifiers)

def run_digits_task(
    test_size=0.2,
    random_state=42,
):
    print("=== DIGITS DATASET ===")

    # 1. Загрузка данных
    digits = load_digits()
    X = digits.data
    y = digits.target

    print("X shape:", X.shape)
    print("y shape:", y.shape)

    # 2. Train / test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=test_size,
        random_state=random_state
```

```

)

# 3. Hyperopt-sklearn (TPE)
estim = HyperoptEstimator(
    classifier=custom_any_classifier("clf"),
    preprocessing=any_preprocessing("prep"),
    algo=tpe.suggest,
)

estim.fit(X_train, y_train)

# 4. Оценка
y_pred = estim.predict(X_test)

print("\n=== METRICS ===")
print("Accuracy:", accuracy_score(y_test, y_pred))

disp = ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion Matrix (Digits, TPE)")
plt.tight_layout()
plt.savefig("cm_digits_tpe.png")
plt.close()

print("\n=== BEST MODEL ===")
print(estim.best_model())

# 5. Без параметров (по умолчанию)
estim_default = HyperoptEstimator()
estim_default.fit(X_train, y_train)
y_pred_def = estim_default.predict(X_test)

print("\n=== DEFAULT METRICS ===")
print("Accuracy:", accuracy_score(y_test, y_pred_def))

disp = ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion Matrix (Digits, TPE)")
plt.tight_layout()
plt.savefig("cm_digits_default.png")
plt.close()

print("\n=== BEST DEFAULT MODEL ===")
print(estim_default.best_model())

def run_electricity_prices_task(
    dataset_id=151,
    test_size=0.2,
    random_state=42,
):
    print("=== ELECTRICITY PRICES DATASET ===")

    # 1. Загрузка данных
    dataset = fetch_openml(data_id=dataset_id, as_frame=True)
    df = dataset.data.copy()
    df["target"] = dataset.target

    print("\n=== FIRST 10 ROWS (FULL DATASET) ===")

```

```

print(df.head(10))

X = dataset.data
le = LabelEncoder()
y = le.fit_transform(dataset.target)

# 2. Описание
print("\n=== DESCRIPTION ===")
print(dataset.DESCR)

print("\n=== INFO ===")
print(X.info())

# 3. Train / test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=test_size,
    random_state=random_state
)

# 4. TPE (как в задании 1)
estim_tpe = HyperoptEstimator(
    classifier=any_classifier("clf"),
    preprocessing=any_preprocessing("prep"),
    algo=tpe.suggest,
)

estim_tpe.fit(X_train, y_train)
y_pred_tpe = estim_tpe.predict(X_test)

print("\n=== TPE RESULTS ===")
print("Accuracy:", accuracy_score(y_test, y_pred_tpe))

disp = ConfusionMatrixDisplay.from_predictions(y_test, y_pred_tpe)
plt.title("Confusion Matrix (Electricity, TPE)")
plt.tight_layout()
plt.savefig("cm_electricity_tpe_2.png")
plt.close()

print("\n=== BEST MODEL ===")
print(estim_tpe.best_model())

# 6. Смешанный алгоритм поиска
mix_algo = partial(
    mix.suggest,
    p_suggest=[
        (0.25, rand.suggest),
        (0.25, anneal.suggest),
        (0.50, tpe.suggest),
    ]
)

estim_mix = HyperoptEstimator(
    classifier=any_classifier("clf"),
    algo=mix_algo,
)

```

```

estim_mix.fit(X_train, y_train)
y_pred_mix = estim_mix.predict(X_test)

print("\n=== MIXED SEARCH RESULTS ===")
print("Accuracy:", accuracy_score(y_test, y_pred_mix))

disp = ConfusionMatrixDisplay.from_predictions(y_test, y_pred_mix)
plt.title("Confusion Matrix (Electricity, MIX)")
plt.tight_layout()
plt.savefig("cm_electricity_mix_2.png")
plt.close()

print("\n=== BEST MODEL ===")
print(estim_mix.best_model())

# 7. Классификатор best default
best = estim_mix.best_model()
learner = best["learner"]

default_clf = learner.__class__()
default_clf.fit(X_train, y_train)
y_pred_def = default_clf.predict(X_test)

print("\n=== ESTIMATOR ===")
print("Accuracy:", accuracy_score(y_test, y_pred_def))

disp = ConfusionMatrixDisplay.from_predictions(y_test, y_pred_def)
plt.title("Confusion Matrix (Electricity, MIX)")
plt.tight_layout()
plt.savefig("cm_electricity_default_2.png")
plt.close()

def run_creditcard_task(
    file_path="credit.arff",
    test_size=0.2,
    random_state=42,
):
    print("=== CREDIT CARD DATASET ===")

    # 1. Загрузка данных из ARFF
    data, meta = arff.loadarff(file_path)
    df = pd.DataFrame(data)

    print("\n=== INFO ===")
    print(df.info())
    print(df.describe())

    # 2. Преобразование данных
    X = df.iloc[:, :-1]
    y = df.iloc[:, -1]

    # 3. Train / test split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y,
        test_size=test_size,
        random_state=random_state

```

```

)

# 4. Hyperopt-sklearn (TPE)
estim = HyperoptEstimator(
    classifier=any_classifier("clf"),
    preprocessing=any_preprocessing("prep"),
    algo=tpe.suggest,
    trial_timeout=100,
)

estim.fit(X_train, y_train)

# 5. Оценка
y_pred = estim.predict(X_test)

print("\n=== METRICS ===")
print("Accuracy:", accuracy_score(y_test, y_pred))

disp = ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion Matrix (Credit Card, ATPE)")
plt.tight_layout()
plt.savefig("cm_creditcard_tpe_3.png")
plt.close()

print("\n=== BEST MODEL ===")
print(estim.best_model())

if __name__ == "__main__":
    # run_digits_task()
    # run_electricity_prices_task()
    run_creditcard_task()

```